

節 1 / 共 4 節

從對話到派工

AI 幫你寫程式，跟你問 AI 問題，不是同一件事

這兩天，我們要做一件事

把 LLM 從一個「聊天玩具」，變成一個 **你敢把工作交給它、而且知道怎麼檢查它** 的系統。

重點 這門課不教 transformer、不教訓練、不教數學。我們只做一件事：**把它接進真實的工作流程裡，然後確認它沒騙你。**

LLM 只是個會猜下一個字的東西。
讓它變成能交付的系統的，
是你在它外面搭的回饋迴路。

四節課，就是把迴路搭起來

節次	問題	你會學到
節 1	派工跟問答差在哪？	agent 能做什麼、你要給它什麼
節 2	它有什麼工具與規矩？	harness：AGENTS.md、skill、工具、迴路
節 3	它看得到什麼？	RAG、產品接法、成本與資安
節 4	你憑什麼相信它？	輸出驗證、eval、不要全部 vibe coding

實務 前三節在**放大**它的能力，第四節在**收斂**它的不確定性。只做前面 → 做出一個沒人敢用的東西；只做後面 → 什麼都沒做出來。

這門課怎麼上

講述 4 成、動手 6 成。你會一直在敲鍵盤。

規則

- 每節 3 小時，中間休息 10 分鐘
- 共 7 個 Lab，每個都有**基礎**與**進階**兩層
- 有任何問題可以舉手發問

你需要準備

- 一台 Windows 11 —— 環境等一下當場裝
- 一個 OpenCode Zen 帳號（等一下註冊）
- 一個能記東西的地方（記事本、筆記軟體都行）

注意 免費模型額度有限，不要拿它閒聊，額度要留給 Lab。

\$ whoami

陳冠惟 Guan-Wei Chen

現在

- 陽明交通大學 網路工程研究所 碩二
- Panglotlabs 後端軟體工程師

經歷

- 東海資工 41th
- 東海大學網管小組成員
- 工業技術研究院 實習

證照 & 榮譽

- CEH (Certified Ethical Hacker)
- AIS3 EOF 決賽



1-A

它的邊界，與 agent 怎麼補

先看模型本身做得到什麼、做不到什麼。

「怎麼問」這一節不教

網頁版的使用經驗，已經涵蓋了這一層。

- 問得模糊，答得就模糊
- 給例子比給描述準
- 講清楚格式，它就照做

所以這一節不花時間教你怎麼問。
我們直接跳到你還沒用過的那個東西。

情境一：用 AI 寫 code

問「Rust 怎麼產生亂數」。它照**訓練時看過的資料**回答——而那份資料有時效。

```
``toml
[dependencies]
rand = "0.8"
``

fn main() {
    let mut rng = thread_rng(); // 获取随机数生成器

    // 1. 生成整数 (均匀分布)
    let n: i32 = rng.random();           // 任意 i32
    let n: usize = rng.random();         // usize
    let n: u8 = rng.random();             // u8
    let n = rng.gen_range(1..=10);        // 1 到 10 (包含两端)
```

rand = "0.8" 最新是 0.9.5 | thread_rng() \ gen_range() 0.8 的名字 | **rng.random() 是 0.9 的名字**

一個編不過，一個滿是 deprecated

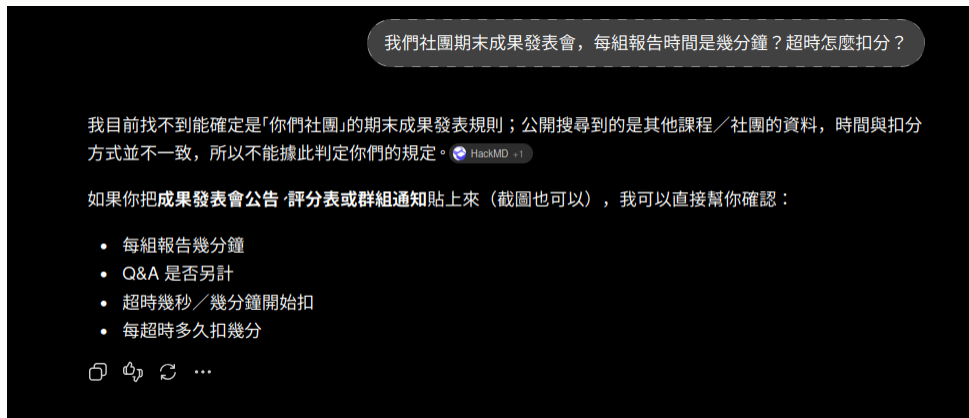
情境一（續）：同一段程式碼，配不同版本。

依賴宣告	cargo build 的結果
rand = "0.8"	✗ 編譯失敗 —— E0599: ThreadRng 沒有 random 這個 method
rand = "0.9"	△ 能編能跑 ，但一整排 deprecated 警告

注意 它混用了兩個版本的 API，讀 code 讀不出來 —— cargo build 一秒就講了。

情境二：叫 AI 處理你自己的東西

問一個社團內部才知道的規定。它去搜尋了——但搜到的是別人的。



重點 它搜到的是別的社團，所以它說：「你把公告貼上來，我可以直接幫你確認。」

注意 搜尋救不了你——這份手冊不在網路上，而且你得先知道答案在哪一份文件。

這門課教的每樣東西，都對應一種你會遇到的狀況

熱門、穩定、通用的東西它很準。會卡住的就這幾種情況。

你想做的事	會遇到什麼	用什麼解
用一個沒用過的套件	版本落後、API 搬家	讓它去查、去跑編譯 節 2
要它照專案的風格寫	它不知道你的規矩	AGENTS.md 節 2
同一套流程一直重跑	每次都要重講一遍	Skill 節 2
要它碰內部系統／私有文件	它拿不到	工具 / MCP、RAG 節 2、3
要它產出程式要吃的資料	算錯、欄位對不上	輸出驗證 節 4

注意 另一個獨立的問題：**同樣輸入不保證同樣輸出**。

補救的東西，全都在模型外面

同樣的模型，agent 多了三件事。

	網頁版	Agent
看得到什麼	公開網路 + 你貼進去的字	你的專案 ，它自己開檔案讀
能做什麼	只能回字（頂多幫你搜尋）	真的動手 ：跑編譯、跑測試、看錯誤
記得什麼	跨 session 要重講一遍	對話一樣不跨 session，但 設定檔每次自動載入

重點 網頁版是問答，**agent 是派工**。你不再是複製貼上的中間人。

實務 **模型沒變，變的是它外面**。接下來三節，就是把這三件事一個一個接上去。

等一下要裝的兩個東西

先講清楚，免得你在往 PowerShell 貼指令的時候心裡發毛。

名稱	
pi	一個跑在你終端機裡的 coding agent。跟 Claude Code、Codex 同一類
OpenCode Zen	一個模型中轉站。有免費額度，所以我們用它

重點 今天學的是概念，不是某個工具的 API。換成 Claude Code、Cursor，這門課的每一句話都還成立 —— 只是名字不一樣。

1-B

Lab 0：環境健檢

後面三節課全部依賴這 35 分鐘。

Lab 0 1. 裝 pi

35 分鐘

目標 到 <https://pi.dev/> 複製對應的指令裝 pi。

Windows 11

```
powershell -c "irm https://pi.dev/install.ps1 | iex"
```

它會順便問你要不要裝 Node —— 說要。

Linux / macOS

```
curl -fsSL https://pi.dev/install.sh | sh
```

```
sixsquare@sixsquare-laptop:~$ curl -fsSL https://pi.dev/install.sh | sh

Pi
Pi Installer
There are many agent harnesses but this one is yours

Install command:
npm install -g --ignore-scripts --min-release-age=0 @earendil-works/pi-coding-agent

Choose an action:
y Install Pi (default)
n Do nothing

Will install Pi.

✓ install complete

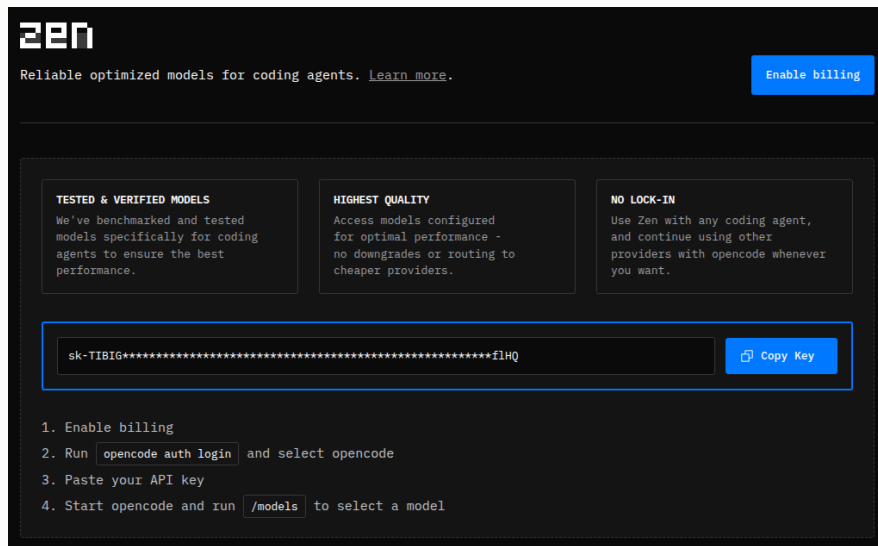
Pi was installed successfully.

Run it with: pi
sixsquare@sixsquare-laptop:~$
```

實務 驗收 開一個新的終端機，打 pi 叫得出來。裝完不換視窗，PATH 不會生效。

Lab 0 2. 拿一把 key

目標 到 OpenCode Zen 註冊，複製 API key。



開 <https://opencode.ai/auth> 註冊／登入，複製 API key。

重點 不需要填信用卡。我們全程只用免費模型。

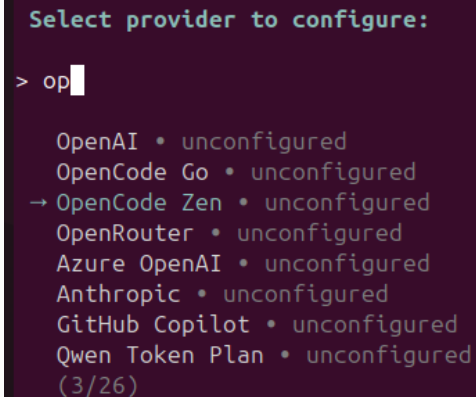
注意 免費模型會記錄你送出的內容。lab 素材都是虛構的——不要餵真實個資。

Lab 0 3. 把 key 接上 pi

目標 在 pi 裡面用 `/login`，不用設環境變數。

1. 終端機打 `pi`，進去之後輸入 `/login`
2. 選 **Sign in with an API key**
3. 選 **OpenCode Zen**
4. 貼上剛剛複製的 key

實務 驗收 pi 開起來，右下角看得到模型名稱，不再出現 `No models available`。

A terminal window with a dark purple background. At the top, it says "Select provider to configure:". Below that, a prompt "> op" is followed by a list of providers, each with a status "unconfigured". The providers are: OpenAI, OpenCode Go, OpenCode Zen (which is highlighted with a blue arrow), OpenRouter, Azure OpenAI, Anthropic, GitHub Copilot, and Qwen Token Plan. At the bottom of the list, it says "(3/26)".

```
Select provider to configure:
> op
OpenAI • unconfigured
OpenCode Go • unconfigured
→ OpenCode Zen • unconfigured
OpenRouter • unconfigured
Azure OpenAI • unconfigured
Anthropic • unconfigured
GitHub Copilot • unconfigured
Qwen Token Plan • unconfigured
(3/26)
```

實務 登入一次就好，今天所有 lab 都走 pi。但**先把 key 貼到記事本**——明天 Lab 6 要直接打 API 時會用到。

Lab 0 4. 確認它會回話

目標 跑通一次，確認可以用。

```
pi --provider opencode --model nemotron-3.5-lightning-free -p "用繁體中文回我一句 hello"
```

一行打完，不要換行 —— PowerShell 不吃 \ 續行。

注意 每次都要指定 `--model`，不然它會用預設模型 —— 那可能是付費的。

實務 驗收 終端機看得到模型的回覆。

備用模型：hy3-free 、 mimo-v2.5-free

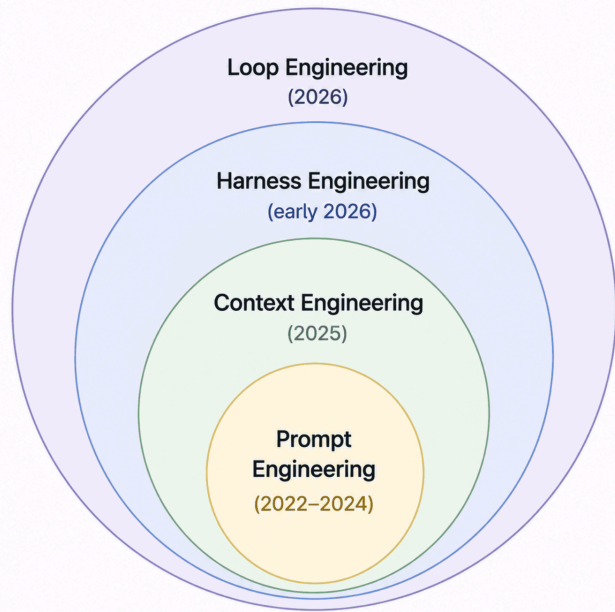
1-C

Agent 時代的四個名詞

prompt / context / harness / loop engineering

這四個不是並列的，是一層包一層

而且它們是照這個順序長出來的——這也是為什麼你只熟第一層。



◎ Each layer contains the one before it.

怎麼讀這張圖

- **外圈把內圈整個包住**——做 loop 的時候，prompt、context、harness 一個都沒有不見
- 你**不是換一種做法**，是在原本的外面再加一層

年份那一欄才是重點

- 2022-2024 大家在調 prompt
- 2025 開始談 context
- **2026 才輪到 harness 與 loop**

所以你只熟最裡面那一圈，很正常——外面兩圈是這兩年才成形的。

同一個任務，四個層次

任務：「幫我把這 **50 份** CSV 整理成月報表」

名詞	白話	在這個任務裡長什麼樣
Prompt engineering	怎麼問	「整理成報表」→「輸出 markdown 表格，欄位 A/B/C，數字兩位小數」
Context engineering	給它看什麼	欄位說明、去年的報表當範例、哪幾欄可以忽略
Harness engineering	給它什麼工具與規矩	讀檔工具、只能寫進 output/、寫完自動跑檢查
Loop engineering	誰決定下一輪、 什麼算完成	一份做完自己挑下一份；而且 欄位邏輯檢查過了才算一份 —— 50 份都通過才停

前兩層你已經在做了

Prompt engineering	把模糊的期待換成明確的規格。給例子比給描述有效， 給反例又更有效
Context engineering	問題常常不是「怎麼問」，是 你根本沒給 。該給相關檔案、規格、範例、先前的決定

注意 這兩層的共通問題：**每次都要重講一遍**。你在網頁版一定有過「又要再解釋一次這個專案」的經驗。

實務 節 2 的第一件事，就是讓這兩層**持久化**——講一次，之後每次都生效。

後兩層是今天開始的新東西

Harness engineering	給它什麼 工具與規矩 ：能讀檔、只能改哪裡、做完要通過什麼檢查
Loop engineering	取代「一直按 enter 的那個人」 ：誰挑下一份工作、 做完了要拿什麼去驗 、什麼時候停

重點 Harness 問「它需要什麼環境」（**一個回合**的事）；
Loop 問「什麼循環讓它朝目標前進、何時該停」（**很多回合**的事）。

實務 Loop 有兩半，**缺一不可**：**挑工作**（下一份 CSV）+ **驗結果**（欄位邏輯對不對）。
只有前面那半，它會很快地產出 50 份錯的東西。

注意 所以停止條件必須**客觀可判定**：**欄位邏輯檢查過了**、編譯過了、測試綠了、清單空了。

多數人花 90% 的時間在調 prompt。
實務上 80% 的效果，
來自後面那三層。

而你已經把 90% 的時間花在第一層了。接下來三節，我們補上另外三層。

1-D

Lab 1：第一次派工

不要再當複製貼上的中間人。

Lab 1 用 agent 做一個真的功能

60 分鐘

目標 先讀規格書，再叫 agent 做，然後**用檢核表驗證**它到底做到了什麼。

```
cd labs/lab1-first-task/habit && node init.ts # git repo + 種子資料
node src/cli.ts list # 先跑一次，翻一下 src/
```

1. 讀 ../SPEC.md (5 分)
2. pi → /model → nemotron-3.5-lightning-free
3. **隨便講**「幫我加一個 streak 指令」，**盯著看它讀了哪些檔** (20 分)
4. git diff → cd .. && node verify.ts → 填第一欄 (10 分)
5. 還原 node restore.ts，**把 SPEC 變成 prompt 再做一次**。兩種給法選一種 —— **A 貼進 prompt** (決定它看到什麼) / **B 寫「先讀 ../SPEC.md」** (決定它去哪找) (25 分)

重點 驗收 第 2 次通過的明顯多於第 1 次。**進階**：換另一種給法，看分數有沒有差。

檢核表：14 項，每項都能用指令確認

node verify.ts 會全部跑一遍 —— 不用憑感覺判斷。

使用方法	不給參數就能跑、列出全部三個習慣
邊界	早起 = 4、 讀論文 = 2（今天沒做，要從昨天算） 、運動 = 0
沒有多做	沒有算「最長連續」
跟上寫法	開在 <code>commands/</code> 、有註冊、用 <code>ui.ts</code> 、用 <code>date.ts</code> 、沒 <code>any</code> 、沒多餘 <code>import</code>

重點 第二次的 **prompt** 要你自己想。規格書已經寫得很清楚了 —— 看第一次哪幾項沒過，那些就是要補進 `prompt` 的東西。

看著它做，不等於知道它做了什麼

所以 `git diff` 那一步不能跳。

畫面上跑過去的是**它想讓你看到的摘要**。 `\ git diff` 是**它實際做了什麼**。

- 它會說「已新增 streak 指令」—— 但不會說它把「目前連續」做成了「最長連續」

注意 這是全課第一次出現「不要相信敘述，去看產出」。節 4 整節都在講這件事。

實測結果：差距全在規格層

同一個模型，同一個專案，兩種講法。

講法	規格層 (8)	風格層 (6)	主要問題
「幫我加一個 streak 指令」	0	5	要吃參數、算成「最長」連續而不是「目前」
照規格講	8	5	只剩一個沒用到的 import

重點 風格層本來就 5/6，兩次一樣。全部的差距都在規格層：0 → 8。

注意 而這些看它跑的時候完全看不出來——它只會說「已新增 streak 指令」。

你第二次補進去的那些話，有兩種

只有一種需要寫成檔案。

這次任務的規格	這個專案的規矩
「不吃參數，列出全部」	「用 <code>date.ts</code> 的 helper」
「邊界這樣算」	「不要多做沒要求的功能」
下次做別的功能就用不到	每次都一樣

- 左邊那堆本來就該每次打 —— 那是 prompt 的工作
- 右邊那堆**每次都一樣**，卻要你每開一個新對話就重打一遍

重點 下一節第一件事：**把右邊那堆寫成檔案，讓 agent 每次啟動就自動載入。**

收斂：context 的四個來源

Lab 1 之後，我們把「該給什麼」整理成四類。

來源	內容	誰決定	在哪一節
你當下打的字	這次的具體要求、格式、邊界	你	節 1
你指給它的檔	「讀 ../SPEC.md」	你	節 1
工具自己抓的	它自作主張去 ls、去 read	它	節 1 已用
常駐的設定檔	專案規範、慣例、禁止事項	你	節 2
外部知識庫	文件、wiki、內部規定	你	節 3

重點 前兩種是你做的 context engineering，第三種是它替你做的。

實務 後兩種是接下來兩節的事：讓「你決定的」那部分**不用每次重打**（節 2）、**大到塞不下也撈得到**（節 3）。

四個層次 vs 四節課，不是一對一

剛剛那張同心圓是「概念的順序」，這門課走的是「做得到的順序」。

層次	這門課在哪裡教	做在哪個 Lab
Prompt	節 1：把規格變成 prompt	Lab 1
Context	被拆成兩半 —— 持久的那半在節 2（AGENTS.md），檢索的那半在節 3（RAG）	Lab 2、Lab 5
Harness	節 2：工具、規矩、守衛	Lab 2、3、4
Loop	也被拆成兩半 —— 迴路機制在節 2，「什麼算完成」在節 4	Lab 4、Lab 7

注意 所以你不會有一節課叫「context engineering」。它是分散的 —— 因為「該給什麼」這件事，在有工具跟沒工具的時候答案完全不同。

你有 agent 了，但它還不知道你的規矩

節 1 小結

你今天學到

- agent 是派工，不是問答
- 它的錯誤是有規律的
- 你覺得理所當然的，它不知道
- prompt / context / harness / loop

你今天做到

- 跑起了自己的 coding agent
- 用它做出一個真的功能
- 跑出了 14 項檢核的兩組數字

下一節：Harness Engineering

開始動模型**外面**的東西：

- 每次開機就知道專案規矩
- 給它工具去碰外部系統
- 做完自動被檢查

實務 你第二次寫的那段 prompt，下一節會變成 agent **每次啟動都自動載入**的檔案。